Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель С.В.Умбетов

«___» _____ 2026 г.

Отчет по лабораторной работе №
по дисциплине «Устройство и функционирование интеллектуальных информационных систем»
«Статистическая генерация текста на основе марковских цепей»

ЛР 09.02.08.19.004

Студент группы 1ИИС-41 Н.Ю. Потапов
группа инициалы., фамилия
Преподаватель старший преподаватель, доцент С.В. Умбетов
должность, ученая степень инициалы., фамилия

Барнаул 2026

Цель работы: Изучить основы статистической генерации текста на основе цепей Маркова, освоить работу с библиотекой markovify, понять влияние порядка модели на качество и связность генерации, а также научиться комбинировать различные текстовые стили.

Задачи к работе:

1. Инициализировать окружение и импортировать необходимые библиотеки (markovify, random).
2. Сформировать исходный обучающий корпус текстов.
3. Построить модель Маркова первого порядка (state_size=1) и сгенерировать предложения.
4. Построить модель Маркова второго порядка (state_size=2) и сгенерировать предложения.
5. Выполнить самостоятельные задания по оценке связности, интеграции кастомного корпуса и комбинированию стилей.
6. Ответить на контрольные вопросы.

Задание принял:  Потапов Н.Ю.

Подпись ФИО

Ход работы:

Задание 1. Инициализация окружения и импорт зависимостей

Ход выполнения: Была произведена установка внешней библиотеки markovify с помощью команды !pip install markovify, так как она не входит в стандартную поставку Python. Также были импортированы модуль markovify для создания текстовых графов переходов и встроенный модуль random для стохастического выбора путей при генерации текста.

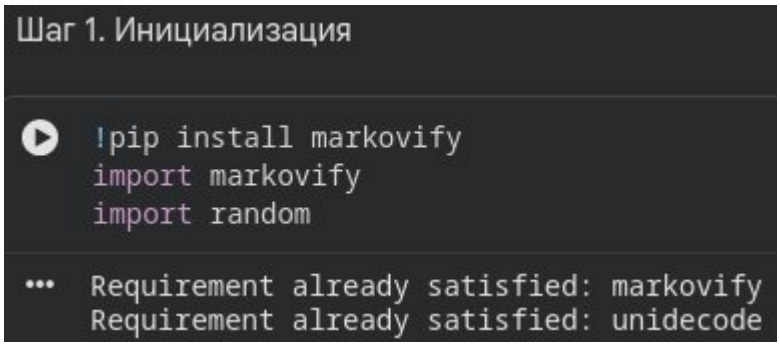


Рисунок 1. Установка библиотеки markovify и импорт зависимостей

Задание 2. Формирование обучающего корпуса

Ход выполнения: Была инициализирована строковая переменная corpus, содержащая фрагмент повести М.Ю. Лермонтова «Герой нашего времени» (описание Пятигорска и окрестностей Машука). Данный текст выступает в роли эмпирической базы данных для расчета частотных характеристик распределения слов. С помощью функции len() была вычислена общая длина строки — 1122 символа, что подтвердило достаточный объем выборки для обучения модели.

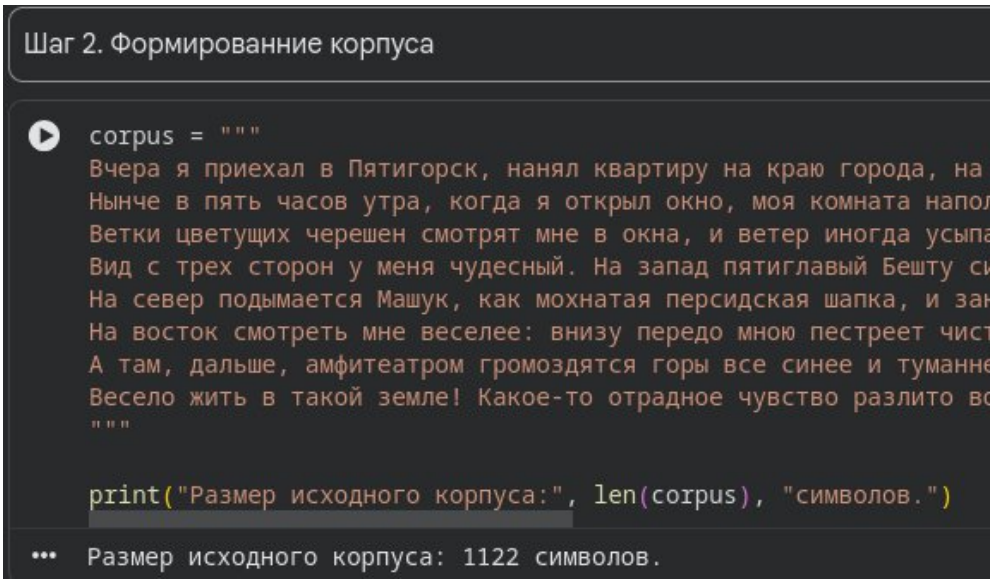


Рисунок 2. Формирование исходного корпуса из произведения М.Ю. Лермонтова

Задание 3. Моделирование случайного процесса первого порядка

Ход выполнения: С помощью класса markovify.Text и параметра state_size=1 была создана модель Маркова первого порядка. Алгоритм автоматически сегментировал текст на предложения, провел токенизацию и построил матрицу вероятностей переходов на основе биграмм. Затем с помощью цикла и метода make_sentence(tries=100) была осуществлена генерация трех независимых предложений.

Примеры сгенерированных фраз:

Фраза 1: Вид с трех сторон у подошвы Машука.

Фраза 2: Ветки цветущих черешен смотрят мне в окна, и свеж, как поцелуй ребенка, солнце светит ярко, небо синее - чего бы, кажется, больше?

Фраза 3: Вид с трех сторон у подошвы Машука.

Параметр tries=100 указывает максимальное количество попыток перезапуска генерации, если цепочка зашла в тупик или слишком сильно дублирует оригинал.

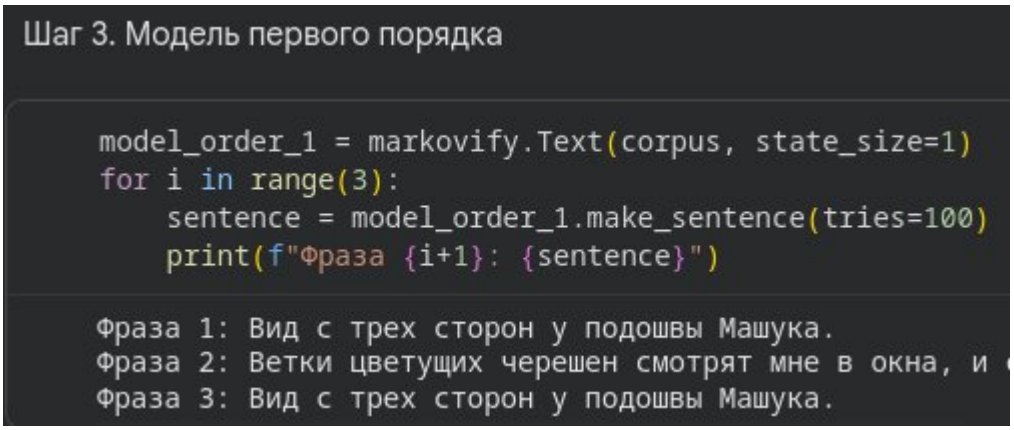


Рисунок 3. Генерация предложений моделью первого порядка (state_size=1)

Задание 4. Моделирование случайного процесса второго порядка

Ход выполнения: Была создана вторая модель с параметром state_size=2, что перевело алгоритм на модель второго порядка. Граф строится на основе триграмм, а условная вероятность выбора последующего токена определяется парой предшествующих слов. При запуске генерации все три попытки вернули значение None.

Результаты:

- Фраза 1: None
- Фраза 2: None
- Фраза 3: None

Причина: из-за малого объема корпуса (1122 символа) и жесткого контекстного окна в 2 слова граф переходов оказался слишком разреженным — алгоритм не смог сформировать предложение, не дублирующее оригинал, за отведенные 100 попыток.

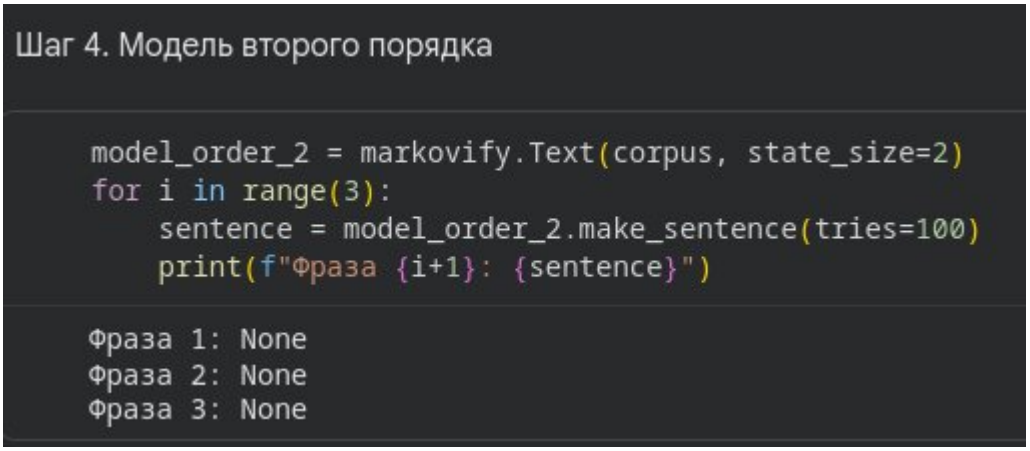


Рисунок 4. Генерация предложений моделью второго порядка (state_size=2)

Задание 5. Самостоятельные задания

5.1. Оценка влияния параметра связности (state_size)

Ход выполнения: Был дописан код, генерирующий 10 предложений для модели со state_size=1 и 10 предложений для модели со state_size=2.

Результаты модели первого порядка:

- На запад пятиглавый Бешту синеет, как поцелуй ребенка, солнце светит ярко, небо синее - чего бы, кажется, больше?
- Ветки цветущих черешен смотрят мне в Пятигорск, нанял квартиру на самом высоком месте, у подошвы Машука.
- Вчера я приехал в Пятигорск, нанял квартиру на краю города, на самом высоком месте, у меня чудесный.
- Вид с трех сторон у подошвы Машука.

Результаты модели второго порядка: все 10 попыток вернули None.

Результаты оценки:

Модель первого порядка (state_size=1) генерирует грамматически менее корректный текст, часто встречаются нарушения согласования слов («Ветки смотрят мне в Пятигорск»), но предложения оригинальны и не являются точной копией исходника.

Модель второго порядка (state_size=2) на малом корпусе не смогла сгенерировать ни одного валидного предложения из-за защитного механизма markovify, отклоняющего точные копии оригинала.

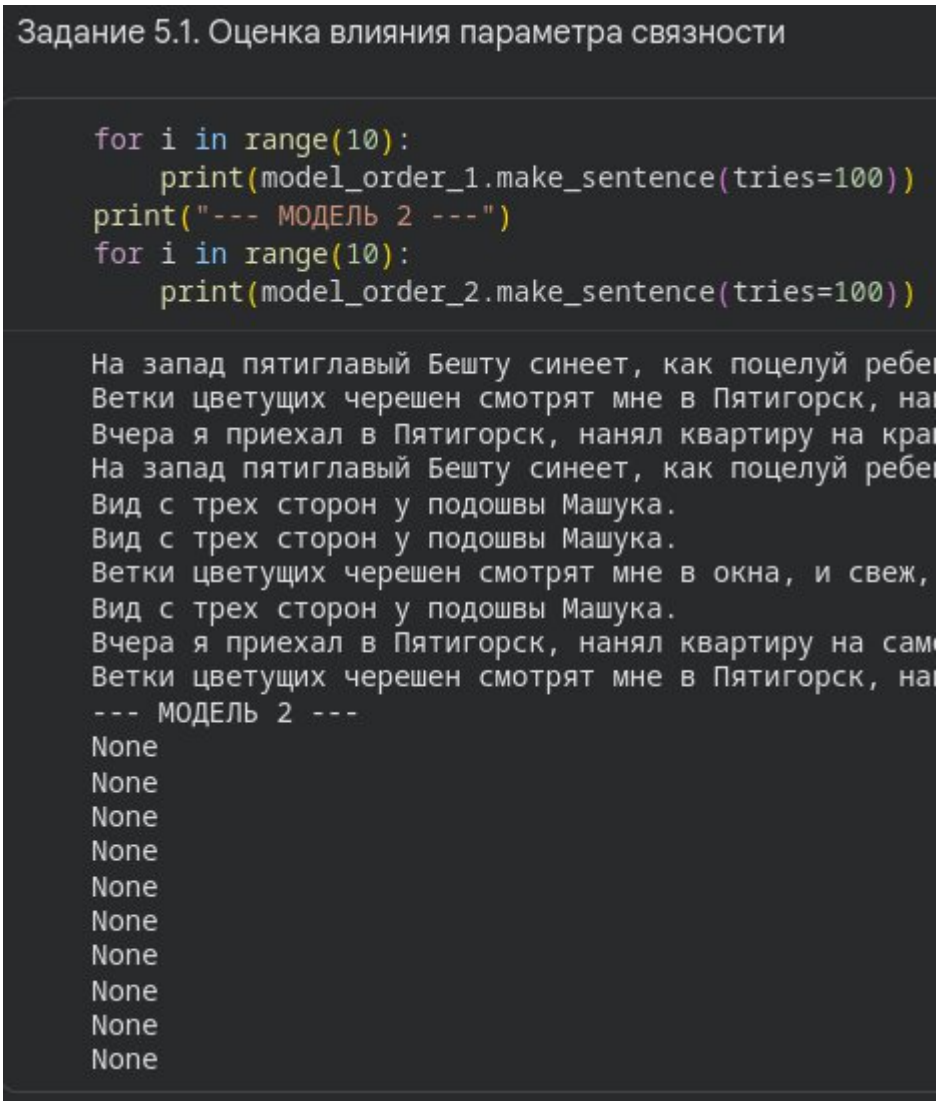


Рисунок 5. Сравнение генерации 10 предложений для моделей state_size=1 и state_size=2

5.2. Интеграция кастомного корпуса и обработка исключений

Ход выполнения: Демонстрационный текст был заменен на массив данных объемом более 50 строк — статья про Arch Linux и его сравнение с Windows. На корпусе была обучена модель со state_size=2.

Результаты:

- Без tries: Во-вторых, управление пакетами и доступ к улучшениям и исправлениям.
- С tries: Во-вторых, управление пакетами и доступ к улучшениям и исправлениям.

Объяснение: В данном случае модель успешно сгенерировала предложение даже без параметра tries, так как объем корпуса оказался достаточным для построения связного графа переходов. Однако на практике при малых корпусах метод make_sentence() без дополнительных параметров часто возвращает None.

Это происходит по двум причинам:

1. Библиотека отклоняет фразы, которые слишком сильно дублируют оригинал (защита от банального копирования);
2. В графе переходов возник тупик — нет пути к терминальному токену END. Параметр tries решает эту проблему, увеличивая количество попыток перестроить цепочку слов.

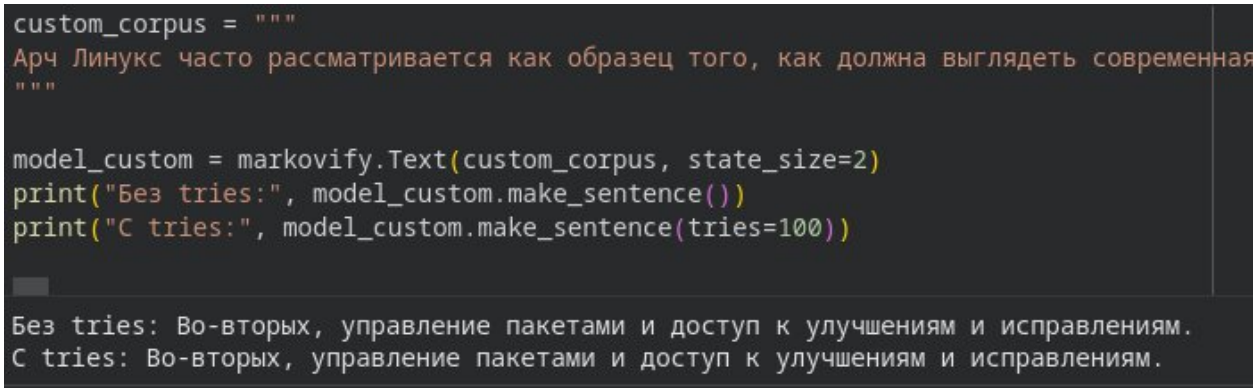


Рисунок 6. Интеграция кастомного корпуса про Arch Linux и генерация предложений

5.3. Комбинирование стилей (Генеративный синтез)

Ход выполнения: Были созданы два разных текстовых объекта: corpus_A (обзор глобальных финансовых рынков, инвестиции, фондовые индексы, криптовалюты) и corpus_B (сказка о Лее и Сердце Леса). На них были обучены две отдельные модели. С помощью встроенной функции markovify.combine модели были слиты с равными весами [0.5, 0.5].

Примеры сгенерированных фраз:

- Сектор цифровых активов также претерпевает значительные изменения на фоне геополитической напряженности и регуляторных рисков.
- Японский рынок переживает ренессанс на фоне геополитической напряженности и регуляторных рисков.
- Жила-была в далёком краю девочка по имени Клык, охранявший вход в Лес Нежности.
- По словам бабушки, там спрятано Сердце Леса — там, где живут добрые дела, а свет Сердца Леса — камешек, который светится, когда в сердце людей живёт искренняя доброта.

Результат: Получился смешанный стиль, в котором деловая лексика и синтаксис финансовых новостей органично переплетаются со сказочными сюжетами и образами. Особенно показателен пример «Жила-была в далёком краю девочка по имени Клык, охранявший вход в Лес Нежности» — сказочная завязка сочетается с именем персонажа-белки из сказки, что создает комичный и сюрреалистичный эффект.

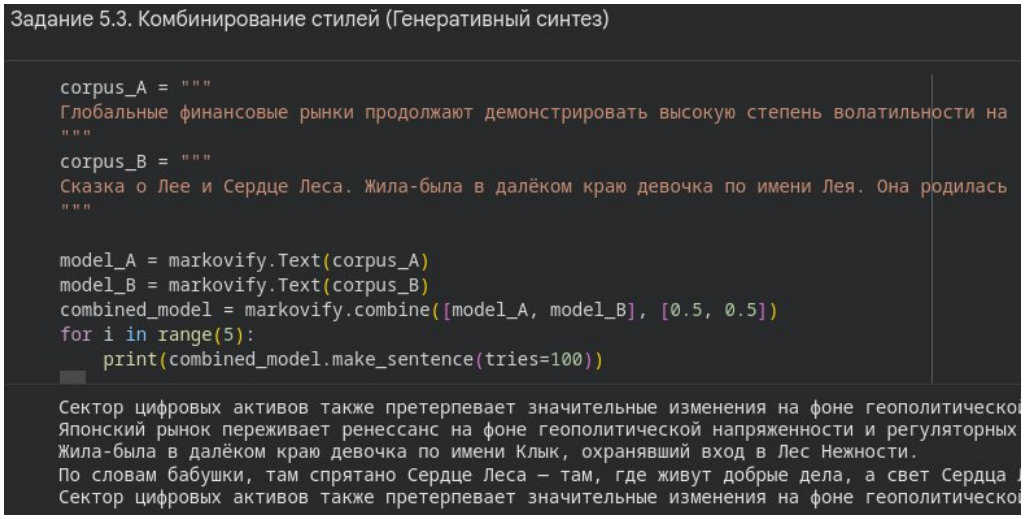


Рисунок 7. Комбинирование двух моделей с весами [0.5, 0.5] и генерация гибридного текста

Вывод:

В ходе лабораторной работы я изучил принципы статистической генерации текста на основе цепей Маркова и освоил работу с библиотекой markovify. На практике убедился, что увеличение параметра state_size повышает грамматическую связность текста, но снижает его оригинальность и увеличивает риск точного копирования фрагментов из обучающей выборки. Эксперимент с малым корпусом Лермонтова показал, что модель второго порядка может полностью отказываться от генерации из-за защитного механизма библиотеки, тогда как модель первого порядка выдает оригинальные, но грамматически некорректные фразы. Научился обрабатывать исключения при генерации через параметр tries и комбинировать модели для создания гибридных текстовых стилей. Понял фундаментальные ограничения марковских моделей, связанные с отсутствием долгосрочной памяти и неспособностью улавливать сюжетный контекст, что отличает их от современных нейросетевых языковых моделей.

Ссылка на мой репозиторий с работами

Ссылка на блокнот данной лабораторной

Ответы на контрольные вопросы:

1. Основное марковское свойство (отсутствие памяти) применительно к генерации текста означает, что вероятность выбора следующего слова зависит исключительно от предыдущего состояния (одного слова или N-граммы) и не зависит от всей предыдущей истории сгенерированной цепочки.

2. N-грамма — это непрерывная последовательность из N элементов (слов) в тексте.

Примеры для фразы «Весело жить в такой земле»:

Униграмма (N=1): «Весело», «жить», «в», «такой», «земле».

Биграмма (N=2): «Весело жить», «жить в», «в такой», «такой земле».

Триграмма (N=3): «Весело жить в», «жить в такой», «в такой земле».

1. Увеличение параметра state_size приводит к росту объема используемой оперативной памяти, так как матрица переходов становится многомерной и содержит больше уникальных комбинаций. При этом связность фраз увеличивается, так как модель учитывает более широкий контекст при выборе следующего слова.

2. Марковские цепи не способны улавливать долгосрочный сюжетный контекст, потому что их память ограничена размером N-граммы (state_size). Они помнят только последние 1, 2 или 3 слова и не могут отслеживать тему абзаца, имена персонажей или развитие сюжета на больших дистанциях.

3. Результат None при вызове make_sentence() означает, что алгоритм не смог сформировать валидное предложение за отведенное количество попыток. Это происходит, если граф переходов не содержит пути к терминальному токену (возник тупик) или если все сгенерированные цепочки были отклонены из-за слишком сильного дублирования оригинала. Предотвращается это увеличением параметра tries или расширением обучающего корпуса.

4. Скрытые токены START и END используются для четкого обозначения границ предложения. START задает начальное состояние для генерации, а END служит терминальным символом, сигнализирующим алгоритму о необходимости завершить случайное блуждание по графу и вывести готовое предложение.

5. Функция markovify.combine позволяет управлять стилистическим весом через передачу списка весов. При генерации алгоритм будет выбирать пути из моделей согласно заданным пропорциям, позволяя гибко смешивать лексику и синтаксис разных корпусов.

6. При state_size=1 модель опирается только на одно предыдущее слово. Русские слова имеют сложную систему флексий, и для выбора правильного окончания часто нужен контекст из двух или трех слов (например, род и число существительного для согласования с прилагательным). Без этого контекста модель выбирает окончания случайно, что приводит к грамматическим ошибкам.

7. Фундаментальное отличие статистического графа Маркова от нейросетевых моделей (LLM) заключается в том, что марковская модель оперирует жесткими частотными вероятностями N-грамм без построения глубоких семантических представлений. Нейросети понимают смысл, контекст и синтаксис на уровне многомерных векторов, тогда как цепи Маркова работают исключительно на уровне поверхностной статистики совместного появления токенов.

8. Объем и жанровое разнообразие корпуса напрямую влияют на вариативность матрицы: чем больше и разнообразнее тексты, тем больше уникальных N-грамм и путей в графе переходов. Это снижает вероятность тупиков, повышает оригинальность генерации и позволяет модели охватывать более широкий спектр лексических и синтаксических конструкций.